

Rapport Traduction des langages

Mediani Hocine, Krief Benjamin

Table des matières :

1. Présentation -----	3
2. Evolution de l'AST -----	4
3. Jugements de typage -----	6
4. Traitement des pointeurs -----	7
5. Traitement des procédures -----	8
6. Traitement du passage par référence -----	9
7. Traitement des types énumérés -----	10
8. Conclusion -----	11

I. Présentation

Ce rapport a pour but de présenter le compilateur RAT implémenté lors des séances de TP, puis en autonomie.

RAT est un langage impératif traitant des booléens, des entiers et des rationnels. Il possède une syntaxe simple se rapprochant du C et fonctionne par compilation en langage TAM, lui-même proche de l'assembleur et s'exécutant dans la machine TAM.

Par défaut, RAT ne permet pas de traiter de manière extensive le paradigme de programmation impérative. Il reste un langage simple, sans gestion de procédures, pourtant essentielles, sans utilisation de pointeurs, sans types énumérés.

Ainsi, nous présenterons dans ce rapport les modifications effectuées aux différentes passes du compilateur RAT et à l'AST en découlant pour pouvoir finalement traiter des points abordés au paragraphe précédent.

II. Evolution de l'AST

Afin de pouvoir traiter les différents fonctionnalités que nous souhaitons implémenter, il nous faut changer de nombreuses choses :

- Les types existants,
- Le lexer,
- Le parser,
- La structure des AST à la suite de chaque passe,
- Les passes en elles-mêmes.

Focalisons nous sur les types et la structure de l'AST pour l'instant.

1. Evolution des types

Initialement, les types présents en RAT étaient les suivants :

```
type typ = Bool | Int | Rat | Undefined
```

Il nous faut alors en rajouter 3, spécifiquement pour le traitement des pointeurs, des procédures et des énumérations. Ces nouveaux types sont constitués :

- d'un type pointeur vers un type déjà existant,
- d'un type void pour les fonctions n'ayant que des effets de bord (type print ou modification de variables),
- et d'un type spécifique pour les enregistrements.

Nos nouveaux types enrichis sont alors :

```
type typ = Bool | Int | Rat | Ptr of typ | Void | TID of string  
Undefined
```

Grâce à ces modifications, nous pouvons alors concevoir des pointeurs, des pointeurs de pointeurs, des procédures, des énumérations, et même des pointeurs vers des énumérations, parfait !

Cependant, ces types ne s'attribuent pas tout seul, il faut maintenant changer nos AST et notre parser afin de pouvoir les utiliser pleinement.

2. Evolution de l'AST

Notre compilateur étant enrichi, il en va de soit que notre grammaire en sera changée. Nous allons alors explorer les différents changements effectués à la TDS et l'AST.

a. Changements apportés à la TDS

Actuellement, 3 objets portent des informations en eux :

- Les constantes, en tant qu'**InfoConst**,
- Les variables, en tant qu'**InfoVar**,
- Et les fonctions, en tant qu'**InfoFun**.

Pour ce qu'il en est de la TDS, nous avons fait le choix d'ajouter 2 type d'informations pour faciliter le traitement des énumérations :

- L'**InfoEnum** pour l'énumération entière,
- Et l'**InfoIds** pour les types énumérés.

Ces nouvelles informations nous permettront alors de plus facilement analyser les énumérations lors de la compilation.

b. Changements apportés à l'AST

Nous en venons alors au cœur des changements : l'AST. Initialement, nous obtenons à l'étape de parsing un AstSyntax.

Pour satisfaire à notre langage RAT étendu, il nous faut introduire de nombreux nouveaux types et expressions, tels que les affectables, les appels de procédures, etc. Tout cela sera vu en détail lors des prochaines parties, mais de ces changements, nous obtenons cette nouvelle grammaire (et donc structure d'AST) :

1. $PROG' \rightarrow ENUM^* PROG \$$	5. $A \rightarrow id$
2. $ENUM \rightarrow enum\ tid\ \{IDS\};$	6. $ (* A)$
3. $IDS \rightarrow tid\ \langle, tid \rangle^*$	7. $E \rightarrow id\ (CP)$
4. $PROG \rightarrow FUN^* id\ BLOC$	8. $ [E / E]$
5. $FUN \rightarrow TYPE\ id\ (DP)\ BLOC$	9. $ num\ E$
6. $BLOC \rightarrow \{ I^* \}$	10. $ denom\ E$
7. $I \rightarrow TYPE\ id = E;$	id
$id = E;$	11. $ A$
8. $ A = E;$	12. $ true$
9. $ const\ id = entier;$	13. $ false$
10. $ print\ E;$	14. $ entier$
11. $ if\ E\ BLOC\ else\ BLOC$	15. $ (E + E)$
12. $ while\ E\ BLOC$	16. $ (E * E)$
13. $ return\ E;$	17. $ (E = E)$
14. $ return;$	18. $ (E < E)$
15. $ id\ (CP);$	19. $ (E)$
16. $DP \rightarrow \Lambda$	20. $ null$
17. $ ref? TYPE\ id\ \langle, ref? TYPE\ id \rangle^*$	21. $ (new\ TYPE)$
18. $TYPE \rightarrow bool$	22. $ \& id$
19. $ int$	23. $ ref\ id$
20. $ rat$	24. $ tid$
21. $ TYPE^*$	25. $CP \rightarrow \Lambda$
22. $ void$	26. $ E\ \langle, E \rangle^*$
23. $ tid$	

III. Jugements de typage

Dans le cadre du langage RAT étendu, nous obtenons ces jugements de typage, où Γ représente l'environnement de typage (la TDS) et T représente un type :

Booléens :

$$\Gamma \vdash \text{true} : \text{bool}$$
$$\Gamma \vdash \text{false} : \text{bool}$$

Entiers :

$$\Gamma \vdash n : \text{int}$$

Identificateurs :

$$x : T \in \Gamma$$
$$\Gamma \vdash x : T$$

Opérations sur les entiers/rationnels (+ et *) :

$$\Gamma \vdash e_1 : \text{int}, \Gamma \vdash e_2 : \text{int}$$
$$\Gamma \vdash e_1 : \text{rat}, \Gamma \vdash e_2 : \text{rat}$$
$$\Gamma \vdash e_1 \text{ op } e_2 : \text{int}$$
$$\Gamma \vdash e_1 \text{ op } e_2 : \text{rat}$$

Comparaison :

$$\Gamma \vdash e_1 : \text{int}, \Gamma \vdash e_2 : \text{int}$$
$$\Gamma \vdash e_1 < e_2 : \text{bool}$$

Egalité :

$$\Gamma \vdash e_1 : T, \Gamma \vdash e_2 : T, T \in \{\text{int}, \text{bool}, \text{TID}\}$$
$$\Gamma \vdash e_1 = e_2 : \text{bool}$$

Construction de fraction :

$$\Gamma \vdash e_1 : \text{int}, \Gamma \vdash e_2 : \text{int}$$
$$\Gamma \vdash [e_1/e_2] : \text{rat}$$

Accès numérateur/dénominateur :

$$\Gamma \vdash e : \text{rat}$$
$$\Gamma \vdash \text{num } e : \text{int}$$
$$\Gamma \vdash e : \text{rat}$$
$$\Gamma \vdash \text{denom } e : \text{int}$$

Pointeur null :

$$\Gamma \vdash \text{null} : \text{pointer undefined}$$

Allocation de pointeur :

$$\Gamma \vdash \text{new } T : \text{pointer } T$$

Prise d'adresse :

$$\Gamma \vdash x : T$$
$$\Gamma \vdash \&x : \text{pointer } T$$

Déréférencement :

$$\Gamma \vdash e : \text{pointer } T$$
$$\Gamma \vdash *e : T$$

Énumérations :

$$\text{Info}(X) = \text{InfoIds}(_, \text{NomEnum}, _)$$
$$\Gamma \vdash X : \text{TID}(\text{NomEnum})$$
$$\Gamma \vdash e_1 : \text{TID}(N), \Gamma \vdash e_2 : \text{TID}(N)$$
$$\Gamma \vdash e_1 = e_2 : \text{bool}$$

Return void :

$$T_{\text{attendu}} = \text{void}, \Gamma \vdash e : \text{pointer undefined}$$
$$\Gamma \vdash \text{return } e$$

IV. Traitement des pointeurs

Mais un premier problème se pose alors, comment traiter l'utilisation des pointeurs ?

Une observation simple nous met sur le bon chemin : une variable est ou bien un identifiant vers une variable, une constante ou une fonction, ou bien un pointeur vers une variable.

De cette observation découle alors notre premier changement à l'AST, l'introduction d'Affectables.

Nous allons alors enrichir l'AST avec notre nouveau type affectable, qui sera comme décrit précédemment, tantôt un identifiant, et tantôt un déréférencement de pointeur. Son implémentation ressemblerait alors à ça :

```
type affectable =  
  | Ident of string (* puis info_ast *)  
  | Deref of affectable
```

Nous renvoyons donc maintenant depuis le parser des affectables.

Il semble alors bien venu de pouvoir initialiser, allouer et accéder aux données du pointeur que l'on souhaite créer. C'est pour cela que nous allons également ajouter de nouvelles expressions, permettant alors de manipuler pleinement les pointeurs en RAT : l'initialisation, l'allocation, et l'accès à l'adresse mémoire du pointeur.

```
type expression =  
  | Null  
  | New of typ  
  | Adresse of string
```

Un pointeur est donc analysé pour savoir s' il pointe bien vers une variable existante ou vers null, puis on considère si ce dernier est utilisé en lecture ou en écriture et on génère finalement le code TAM adapté, toujours suivant son utilisation en écriture ou en lecture.

V. Traitement des procédures

On souhaite dès à présent pouvoir également traiter des procédures dans RAT. Pour se faire, nous allons nous baser sur la structure actuelle des fonctions (type de retour, nom, liste de types de paramètres), à la seule différence que le type de retour sera toujours void.

Il nous faut alors ajouter une instruction pour les appels de procédures, modifier l'instruction de retour pour permettre le retour vide et le type void lui-même.

```
type instruction =  
| AppelProcedure of string (* puis info_ast *) * expression list
```

Pour gérer les return vides, on affecte à `return;` un `Retour Null`, par convention, ce qui nous force alors lors de la passe de typage de vérifier le type du return dans un cas spécial que voici :

```
| AstTds.Retour (e, ia) ->  
    let (ne, te) = analyse_type_expression e in  
    let t = recuperer_type ia in  
    if ((est_compatible t te) || (t = Void && te = Ptr  
Undefined))  
    then AstType.Retour (ne, ia)  
    else raise (TypeInattendu (te, t))
```

Et le type void a été introduit dans la seconde partie du rapport.

Nous pouvons dès à présent créer et appeler des procédures, ne produisant que des effets de bord, sur des variables ou des pointeurs, les différents ajouts pouvant être utilisés entre eux !

VI. Traitement du passage par référence

Afin de pouvoir utiliser pleinement les procédures, notamment pour leur capacité à produire des effets de bord, il est nécessaire d'autoriser la modification des variables passées en paramètres. Nous avons donc implémenté le passage par référence, identifié par le mot-clé **ref**.

Pour distinguer si une variable est une référence ou une valeur standard, nous avons modifié le type info de la TDS en y ajoutant un booléen :

```
type info =  
  | InfoVar of string * typ * int * string * bool  
  | InfoFun of string * typ * (typ * bool) list
```

On modifie dans l'AST le type fonction en conséquence :

```
type fonction =  
  Fonction of typ * string * (typ * string * bool) list * bloc
```

Et on ajoute ensuite le cas Reference au type expression pour gérer le passage de paramètres par référence :

```
type expression =  
  | Reference of string
```

Du point de vue du passage mémoire, une référence fera une taille de 1. Tandis que du côté de la génération de code, la modification s'effectue sur l'enregistrement et le chargement de la variable.

Si la variable est une référence, on charge d'abord l'adresse contenue dans la variable, puis on effectue la lecture ou l'écriture à cette adresse mémoire, et donc on permet les effets de bords des paramètres.

Ainsi, à l'aide de ces modifications, toutes variables passées en paramètres d'une fonction ou procédure peuvent avoir des effets de bords autorisés, tant que la variable est indiquée en tant que référence.

VII. Traitement des énumérations

Pour ce qu'il en est des énumérations, ces dernières doivent être déclarées avant les fonctions dans le fichier .rat. On les traite alors de cette manière :

- On obtient à la suite de l'étape de parsing un programme avec une liste d'énumérations (vide ou non) et un ensemble de fonction + un bloc principal,
- On ajoute à la TDS les informations relatives aux identifiants et aux énumérations lors de la passe de gestion d'identifiant,
- Pour le reste, lorsque l'on déclare une variable d'un type énuméré, que l'on compare des variables de type énuméré ou autre, tout cela est pris par le compilateur RAT de base.

On modifie alors le type info de la TDS pour pouvoir manipuler nos énumérations comme suit :

```
type info =  
  | InfoEnum of string * string list  
  | InfoIds of string * string * int
```

On modifie dans l'AST le type programme et on y ajoute les types id et énuméré :

```
type ids = string list  
type enum = Enum of string * ids  
type programme = Programme of enum list * fonction list * bloc
```

Et on permet aux expressions de faire référence aux identifiants de type énuméré :

```
type expression =  
  | TIdent of string
```

Ainsi, par toutes ces modifications que nous avons apportées à notre compilateur, il nous est désormais possible d'utiliser des types énumérés en les déclarant en haut du fichier .rat, avec vérifications de type, opérations élémentaires de comparaisons et création de variables de type énumérés !

VIII. Conclusion

Finalement, ce projet nous aura permis de mieux comprendre la manière dont les compilateurs fonctionnent en en implémentant un par nous-mêmes, et en l'améliorant au fur et à mesure.

Bien que le compilateur RAT étendu ne soit pas à la hauteur de GCC ou autre compilateur bien connu, il a tout de même nécessité un travail extensif de raffinage, d'implémentation et de débogage afin de le rendre opérationnel et aux normes attendues par le projet.

Écrire les tests, ajouter de nouvelles fonctionnalités étapes par étapes et voir que le travail donné depuis des semaines prend forme et fonctionne de manière cohérente avec le reste du langage est une expérience très gratifiante.

Question difficulté, il n'y en avait pas vraiment avec la bonne méthode (avancée à la fin du sujet, par étapes) qui permet alors d'isoler les problèmes et de ne faire face qu'à des soucis de logique d'implémentation, et non de non fonctionnement entre les passes. Cette approche méthodique nous a sûrement fait économiser quelques dizaines d'heures de questions, de doutes et de recherche de bugs en vain.

La prochaine étape serait maintenant de traiter les strings par exemple, type classique mais pourtant manquant au langage RAT, d'ajouter de nouvelles fonctions de base telles que l'exponentiation, et permettre l'utilisation et la création de packages RAT.

Le compilateur RAT étendu peut donc maintenant confortablement traiter des pointeurs, des procédures, des passages par référence et des types énumérés.